KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

파이썬 기초 2
문자열, 리스트, for

주문 세 건을
영수증 한 장으로 찍어요

```
==================================
            KANT 카페
==================================
메뉴명          단가  수량        금액
----------------------------------
coffee        4,500     2       9,000
latte         5,000     1       5,000
cake          6,500     3      19,500
----------------------------------
합계                           33,500
==================================
```

오늘의 순서

1	**문자열과 출력**	자르기, sep과 end, %, format(), f-string, 칸 수, 소수점, 콤마	5–17쪽
2	**리스트, 튜플, for**	[ ]와 ( ), 메서드, split과 join, 주문 데이터, 꺼내 담기, for	18–35쪽
3	**영수증 만들기**	주문 한 줄, for로 영수증 한 장, 주문 메모 정리	36–38쪽
	정리	막힐 때, 언제 무엇을 쓰는지	39–40쪽

준비

VS Code로 hello-uv 열기 · `basics.ipynb` 열기 · 커널 `.venv` 확인 · 셀마다 `Shift` + `Enter`

들어가기

12에서 배운 것만으로 찍으면 칸이 들쭉날쭉해요

```
[1] print("coffee", 4500, 2, 4500 * 2)
    print("latte", 5000, 1, 5000 * 1)
    print("cake", 6500, 3, 6500 * 3)
    print("합계", 4500 * 2 + 5000 * 1 + 6500 * 3)
```

```
coffee 4500 2 9000
latte 5000 1 5000
cake 6500 3 19500
합계 33500
```

글자 길이가 달라서
숫자 칸이 안 맞아요

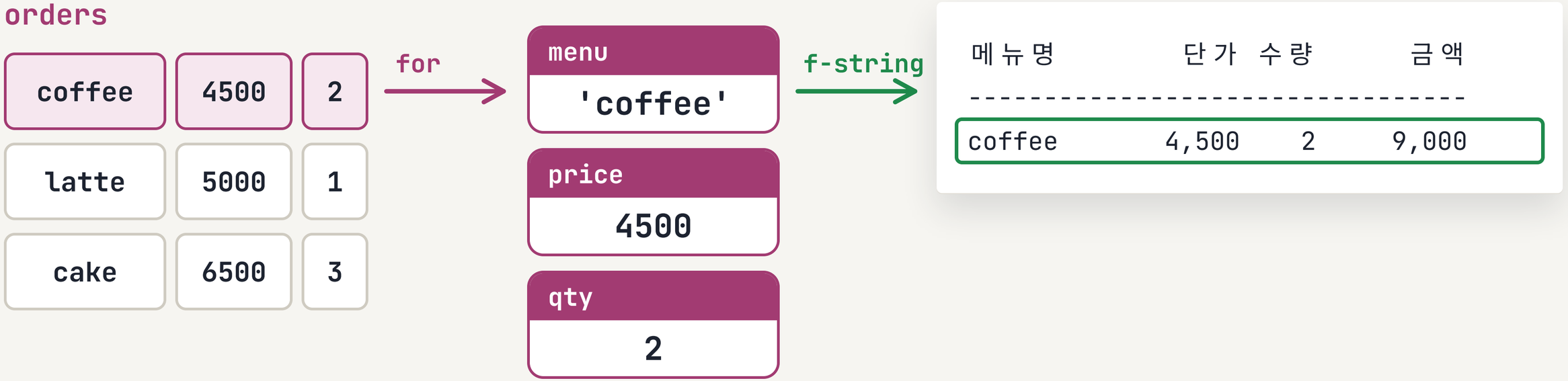
이번 챕터 끝에 찍을 모양

```
메뉴명          단가  수량      금액
-------------------------------
coffee       4,500     2     9,000
latte        5,000     1     5,000
cake         6,500     3    19,500
-------------------------------
합계                        33,500
===============================
```

주문이 열 건이면 print도 열 줄을 써야 해요.

들어가기

주문 데이터가 영수증 한 줄이 되기까지



리스트 하나에 주문 튜플 세 개

한 건씩 꺼내
이름 세 개에 나눠 담기

칸을 맞춰 영수증 한 줄로

1파트 문자열 자르기, f-string으로 칸 맞추기

2파트 리스트, 튜플에 담고 for로 하나씩

3파트 셋을 합쳐 영수증 완성

1. 문자열과 출력

문자열은 따옴표로 감싼 글자예요

[1]
```
a = 'kim'
b = "kim"
print(a == b)
print(len("hello"))
print("It's me")
```

'kim' 와 "kim" 는 같은 값이라 True

글자 안에 ' 가 있으면 바깥을 " 로

```
True
5
It's me
```

h	e	l	l	o
1	2	3	4	5

len() 은 글자 수를 세요

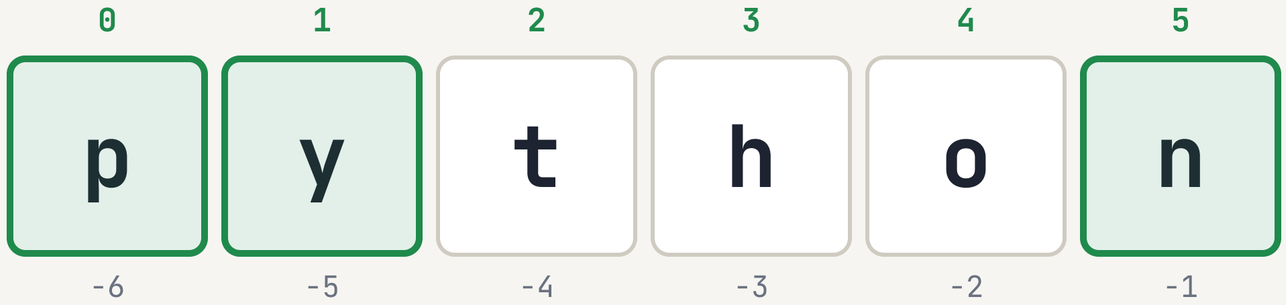
한 파일 안에서는 한 가지 따옴표로 맞춰 쓰면 읽기 쉬워요.

1. 문자열과 출력

[번호]로 글자 하나를 꺼내요. 번호는 0부터예요

[2]
```
word = "python"
print(word[0])
print(word[1])
print(word[-1])
```

```
p
y
n
```



위는 앞에서부터, 아래는 뒤에서부터 센 번호

[3]
```
word[6]
```

```
IndexError: string index out of range
```

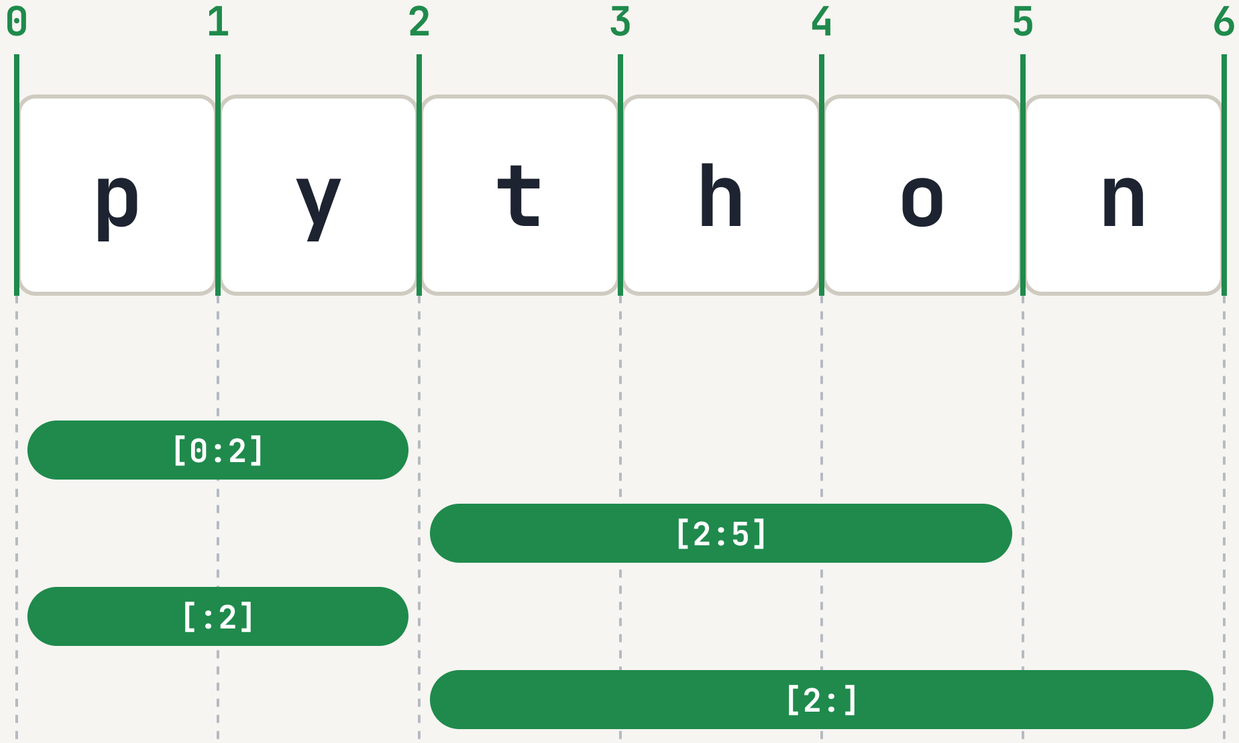
마지막은 5번이라 6번은 없어요

1. 문자열과 출력

[시작:끝]은 글자 사이 번호로 잘라요

```
[4] print(word[0:2])
    print(word[2:5])
    print(word[:2])
    print(word[2:])
```

```
py
tho
py
thon
```



`[0:2]` 는 0번 선부터 2번 선까지라서 두 글자예요. 앞을 비우면 처음부터, 뒤를 비우면 끝까지예요.

1. 문자열과 출력

print의 sep, end와 \n, \t로 출력 모양을 바꿔요

[5]
```
print("kim", "lee")
print("kim", "lee", sep="-")
print("2026", "11", "02", sep=".")
print("hello", end=" ")
print("world")
print("첫째 줄\n둘째 줄")
print("이름\tkim")
```

sep 값 사이에 넣을 글자. 기본은 빈칸

end 줄 끝에 붙일 글자. 기본은 줄바꿈

\n 줄바꿈

\t 탭, 칸 띄우기

```
kim lee
kim-lee
2026.11.02
hello world
첫째 줄
둘째 줄
이름	kim
```

1. 문자열과 출력

값을 끼워 넣는 방법은 세 가지예요

[6]
```
cake = "치즈케이크"
price = 5000
qty = 2
print("주문: %s %d개" % (cake, qty))
print("주문: {} {}개".format(cake, qty))
print(f"주문: {cake} {qty}개")
```

% 포맷팅 가장 옛 방식

format() 파이썬 3.0부터

f-string 3.6부터. 새 코드는 이걸로

```
주문: 치즈케이크 2개
주문: 치즈케이크 2개
주문: 치즈케이크 2개
```

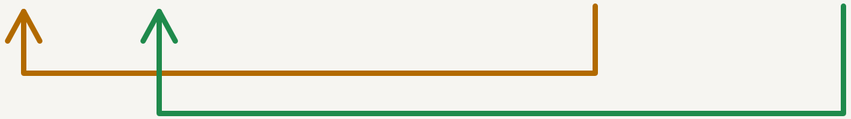
세 줄 모두 결과가 같아요

1. 문자열과 출력

%는 자리 순서대로, f-string은 이름을 바로 적어요

% 포맷팅

```
"%s %d개" % (cake, qty)
```



%s 글자 자리, %d 정수 자리
값은 뒤 괄호에 순서대로

f-string

```
f"{cake} {qty}개"
```

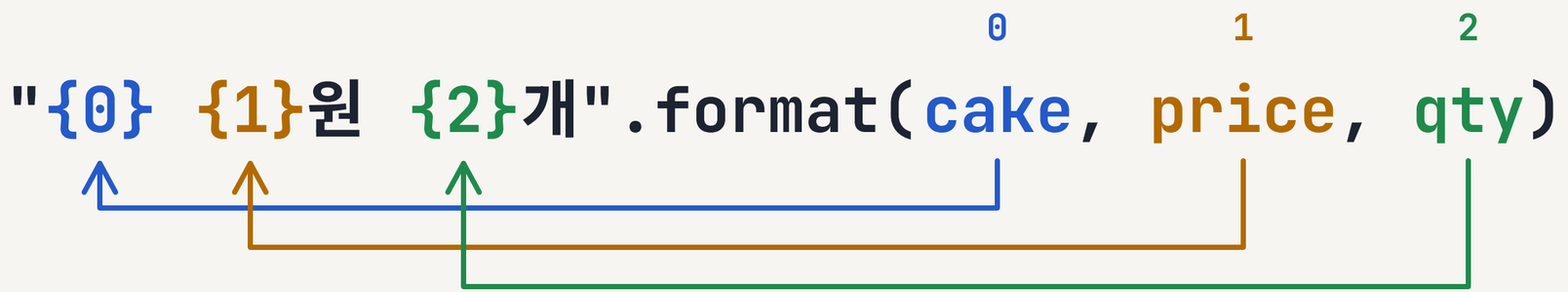
자리에 이름을 바로 적어요
{qty + 1} 처럼 계산식도 돼요

[7]
```
print("%s %d개" % (cake, qty))
print(f"{cake} {qty}개")
print(f"{cake} {qty + 1}개")
```

```
치즈케이크 2개
치즈케이크 2개
치즈케이크 3개
```

1. 문자열과 출력

format()은 {0} {1} {2} 번호로 값을 골라요



[8]
```
print("{0} {1}원 {2}개".format(cake, price, qty))
print("{0} {2}개, 하나에 {1}원".format(cake, price, qty))
print("{0}, {0}".format(cake))
```

번호 순서대로

번호를 바꾸면 순서가 바뀌어요

같은 번호를 여러 번 써도 돼요

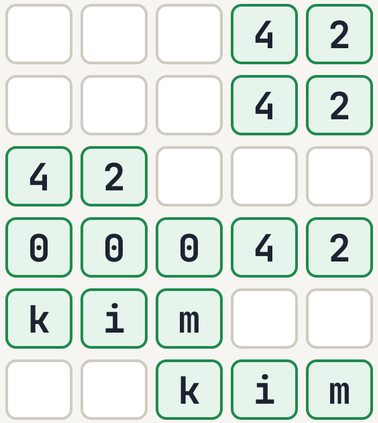
```
치즈케이크 5000원 2개
치즈케이크 2개, 하나에 5000원
치즈케이크, 치즈케이크
```

1. 문자열과 출력

콜론 뒤 숫자는 칸 수예요

[9]
```
print("[%5d]" % 42)
print(f"[{42:5d}]")
print(f"[{42:<5d}]")
print(f"[{42:05d}]")
print(f"[{'kim':5}]")
print(f"[{'kim':>5}]")
```

```
[   42]
[   42]
[42   ]
[00042]
[kim  ]
[  kim]
```



%5d 와 **{42:5d}** 는 같아요

다섯 칸, 숫자는 오른쪽

< 왼쪽 맞춤

0 빈칸을 0으로

글자는 기본이 왼쪽

> 오른쪽 맞춤

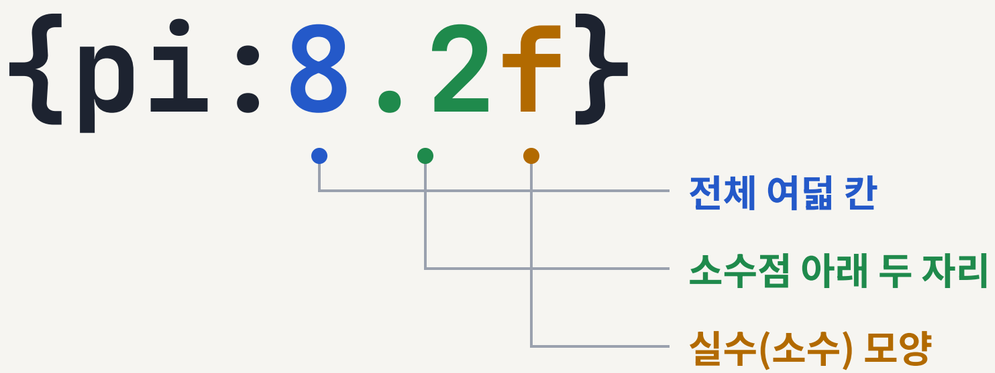
.2f는 소수점 아래 두 자리까지 보여 줘요

[10]
```
pi = 3.141592
print("%.2f" % pi)
print(f"{pi:.2f}")
print(f"{pi:.4f}")
print(f"[{pi:8.2f}]")
```

`%.2f` 와 `{pi:.2f}` 는 같아요

`.4f` 네 자리

```
3.14
3.14
3.1416
[    3.14]
```



1. 문자열과 출력

콜론 뒤 쉼표는 천 단위 콤마예요

[11]
```
price = 1234567
print(f"{price:,}원")
print(f"{1234567.891:,.2f}")
fee = 35000
print(f"[{fee:8,}]")
print(f"{cake} {qty}개, 합계 {5000 * qty:,}원")
```

`:,` 세 자리마다 콤마

`:,.2f` 콤마와 소수 두 자리

`:8,` 여덟 칸과 콤마

계산식 결과에도 붙어요

```
1,234,567원
1,234,567.89
[  35,000]
치즈케이크 2개, 합계 10,000원
```

콤마는 보여 주는 모양만 바꿔요. **price**에 든 값은 그대로 **1234567**이에요.

1. 문자열과 출력

칸 수를 맞추면 줄이 표처럼 맞아요

[12]
```
print(f"{'kim':<6}{90:>5}")
print(f"{'lee':<6}{100:>5}")
print(f"{'park':<6}{7:>5}")
print("%-6s%5d" % ("park", 7))
```

% 방식은 `%-6s` 가 왼쪽 여섯 칸

```
kim        90
lee       100
park        7
park        7
```

이름은 왼쪽 여섯 칸, 점수는 오른쪽 다섯 칸

<6 >5

성적표나 영수증처럼 줄을 맞춰 보여 줄 때 칸 수를 써요.

1. 문자열과 출력

세 방식 모두 콜론 뒤 모양은 같아요

모양	% 방식	format()	f-string	결과
다섯 칸	`%5d`	`{0:5d}`	`{42:5d}`	`[   42]`
소수 두 자리	`%.2f`	`{0:.2f}`	`{pi:.2f}`	`3.14`
천 단위 콤마	없음	`{0:,}`	`{price:,}`	`1,234,567`

[13]
```
print("[{0:5d}]".format(42))
print("{0:.2f}".format(pi))
print("{0:,}원".format(price))
```

format()은 콜론 앞에 번호,
f-string은 이름을 적어요

```
[   42]
3.14
1,234,567원
```

1. 문자열과 출력

자리와 값이 안 맞으면 오류가 나요

```
[14] "%d개" % "kim"
```

```
TypeError: %d format: a real number is required, not str
```

%d 자리에 글자를 넣었어요

%s 로 바꾸거나 f-string으로

```
[15] "%s %d개" % cake
```

```
TypeError: not enough arguments for format string
```

자리는 둘인데 값은 하나예요

% (cake, qty) 처럼 값을 다 적어요

```
[16] f"{cake:d}"
```

```
ValueError: Unknown format code 'd' for object of type 'str'
```

글자에 정수 모양 d를 줬어요

{cake} 만 적어요

```
[17] "%,d" % price
```

```
ValueError: unsupported format character ',' (0x2c) at index 1
```

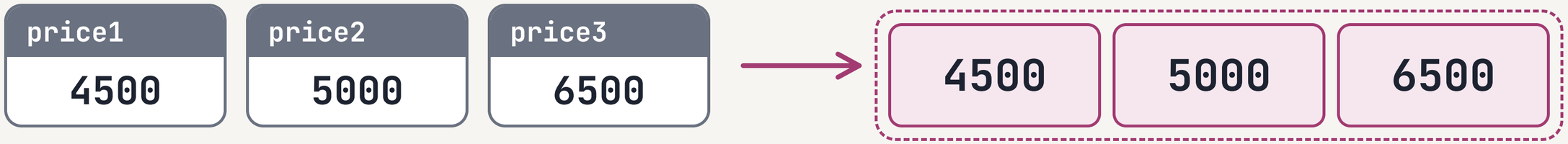
% 방식에는 콤마 옵션이 없어요

f"{price:,}" 로 써요

2. 리스트, 튜플, for

리스트는 값 여러 개를 [ ] 하나에 담아요

12까지는 값마다 변수 하나




```
[1] prices = [4500, 5000, 6500]
    print(prices)
    print(len(prices))
    print(type(prices))
```

- 대괄호 안에 쉼표로 나눠 담아요
- `len()` 은 담긴 개수
- 타입 이름은 `list`

```
[4500, 5000, 6500]
3
<class 'list'>
```

꺼내고 자르는 법은 문자열과 같아요

```
[2] menus = ["coffee", "latte", "cake"]
    print(menus[0])
    print(menus[-1])
    print(menus[0:2])
```

```
coffee
cake
['coffee', 'latte']
```



1파트 문자열과 같은 문법

`word[0]` ↔ `menus[0]`

`word[-1]` ↔ `menus[-1]`

`word[0:2]` ↔ `menus[0:2]`

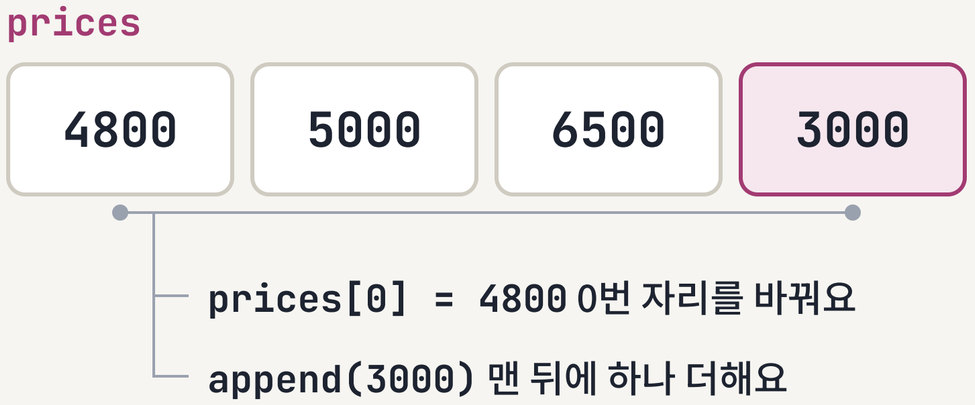
2. 리스트, 튜플, for

리스트는 값을 바꾸고 더할 수 있어요

```
[3] prices[0] = 4800
    prices.append(3000)
    print(prices)
    print(sum(prices))
    print(max(prices))
    print(min(prices))
```

```
[4800, 5000, 6500, 3000]
19300
6500
3000
```

sum 합계, max 가장 큰 값, min 가장 작은 값



넣고 빼는 메서드: append, insert, remove, pop

```
[4] cart = ["coffee", "latte"]
cart.append("cake")
cart.insert(0, "tea")
cart.remove("latte")
last = cart.pop()
first = cart.pop(0)
print(cart, last, first)
```

```
['coffee'] cake tea
```

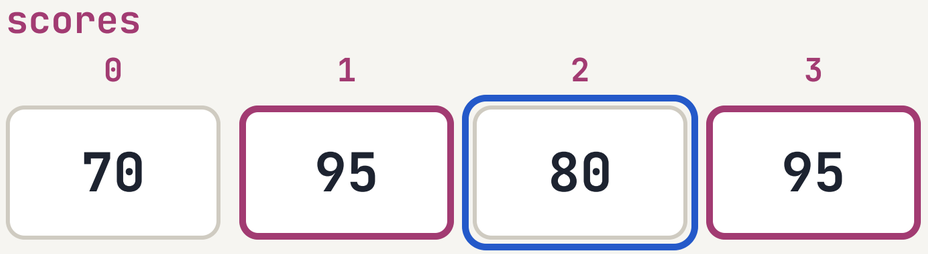
pop은 뺀 값을 돌려줘서 변수에 담을 수 있어요



찾고 세고 정렬하는 메서드: count, index, sort

[5]
```
scores = [70, 95, 80, 95]
print(scores.count(95))
print(scores.index(80))
print(95 in scores)
scores.sort()
print(scores)
scores.sort(reverse=True)
print(scores)
```

```
2
2
True
[70, 80, 95, 95]
[95, 95, 80, 70]
```

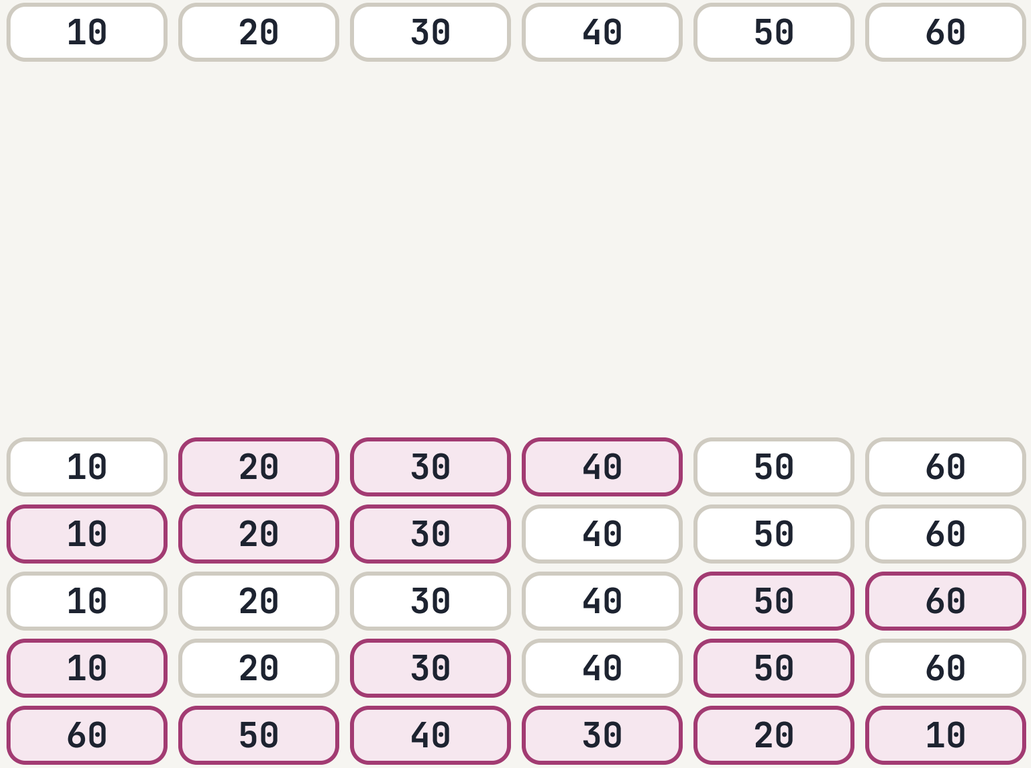


- `count(95)` 95가 몇 개인지
- `index(80)` 80이 몇 번 자리인지
- `95 in scores` 들어 있으면 True
- `sort()` 작은 수부터. 리스트가 바뀌어요
- `reverse=True` 큰 수부터

[시작:끝:간격]으로 원하는 만큼 잘라요

```
[6] nums = [10, 20, 30, 40, 50, 60]
    print(nums[1:4])
    print(nums[:3])
    print(nums[-2:])
    print(nums[::2])
    print(nums[::-1])
```

```
[20, 30, 40]
[10, 20, 30]
[50, 60]
[10, 30, 50]
[60, 50, 40, 30, 20, 10]
```



셋째 숫자는 간격이에요. **2**면 한 칸씩 건너뛰고, **-1**이면 거꾸로 가요.

리스트 안의 리스트는 번호를 두 번 써요

```
[7] table = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(table[1])
print(table[1][2])
print(table[-1][-1])
print(menus[0][0])
mixed = ["coffee", 4500, True]
print(mixed)
```

```
[4, 5, 6]
6
9
c
['coffee', 4500, True]
```

`menus[0][0]` coffee의 0번 글자

글자, 숫자, True를 섞어 담아도 돼요

table

	0	1	2
table[0]	1	2	3
table[1]	4	5	6
table[2]	7	8	9

새 리스트를 만드는 것과 원래 리스트를 바꾸는 것

[8]
```
a = [3, 1, 2]
b = [5, 4]
print(a + b, a * 2)
print(sorted(a), a)
a.extend(b)
a.reverse()
c = a.copy()
c.clear()
print(a, c)
```

```
[3, 1, 2, 5, 4] [3, 1, 2, 3, 1, 2]
[1, 2, 3] [3, 1, 2]
[4, 5, 2, 1, 3] []
```

새 리스트	`a + b` `a * 2` 이어 붙이기
새 리스트	`sorted(a)` 정렬, a는 그대로
원래 것이 바뀜	`a.extend(b)` a 뒤에 b를
원래 것이 바뀜	`a.reverse()` 순서만 거꾸로
새 리스트	`a.copy()` 복사본
원래 것이 바뀜	`c.clear()` 전부 비우기

split은 글자를 리스트로, join은 리스트를 글자로



[9]
```
text = "coffee,latte,cake"
words = text.split(",")
print(words)
print(" / ".join(words))
print("hello world".split())
print(list("abc"))
```

괄호를 비우면 빈칸에서 잘라요

list("abc") 는 한 글자씩 나눠요

```
['coffee', 'latte', 'cake']
coffee / latte / cake
['hello', 'world']
['a', 'b', 'c']
```

문자열과 리스트: 꺼내는 법은 같고, 바꾸기만 달라요

	문자열	리스트
번호로 꺼내기, 자르기	`word[0]` `word[0:2]`	`menus[0]` `menus[0:2]`
개수, 들어 있는지	`len(word)` `'p' in word`	`len(menus)` `95 in scores`
자리의 값 바꾸기	안 돼요 (TypeError)	`names[0] = "park"`
메서드	`split`	`append insert remove pop sort`

[10]
```
names = ["kim", "lee"]
names[0] = "park"
print(names)
word = "python"
word[0] = "P"
```

리스트는 바뀌어요

문자열은 안 바뀌어요

```
['park', 'lee']
TypeError: 'str' object does not support item assignment
```

튜플은 ( )에 묶고, 만든 뒤에는 못 바꿔요

```
[11] order = ("coffee", 4500, 2)
     print(order[0])
     print(order[1] * order[2])
     print(order[1:])
     print(type(order))
```

꺼내고 자르는 건 리스트와 같아요

```
coffee
9000
(4500, 2)
<class 'tuple'>
```




```
[12] order[1] = 5000
```

바꾸기는 안 돼요

```
TypeError: 'tuple' object does not support item assignment
```

주문 한 건, 날짜처럼 묶어 두고 바꾸지 않을 값은 튜플에 담아요.

주문 여러 건은 튜플을 리스트에 담아요

[13]
```
orders = [("coffee", 4500, 2), ("latte", 5000, 1), ("cake", 6500, 3)]
print(len(orders))
print(orders[1])
print(orders[1][0])
print(orders[2][1] * orders[2][2])
```

```
3
('latte', 5000, 1)
latte
19500
```

	[0] 메뉴	[1] 가격	[2] 수량
orders[0]	coffee	4500	2
orders[1]	latte	5000	1
orders[2]	cake	6500	3

앞 번호는 몇째 주문, 뒤 번호는 그 안의 몇째 값이에요.

튜플 값을 변수 여러 개에 한 번에 꺼내 담아요

```
[14] menu, price, qty = orders[2]
print(menu, price, qty)
a, b = "kim", "lee"
a, b = b, a
print(a, b)
print(order.count(4500), order.index(2))
print(list(order))
```

```
cake 6500 3
lee kim
1 2
['coffee', 4500, 2]
```

`a, b = b, a` 두 값을 맞바꿔요

튜플 메서드는 **count**, **index** 두 개
고치려면 **list()**로 바꿔서

`orders[2]`

'cake'	6500	3

menu	price	qty
'cake'	6500	3

왼쪽 이름 수와 튜플 값 수가 같아야 해요.

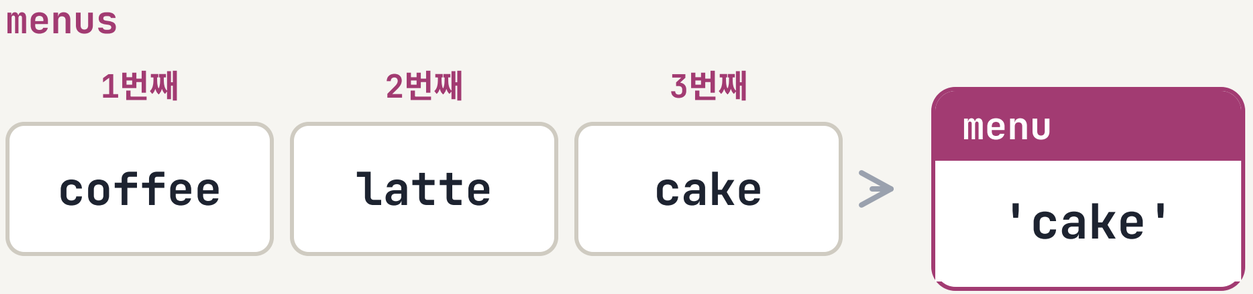
for는 리스트에서 하나씩 꺼내 같은 일을 되풀이해요

[15]
```
for menu in menus:
    print(menu)
```

줄 끝에 콜론 :

네 칸 들여 쓴 줄을 되풀이해요

```
coffee
latte
cake
```



값이 세 개라서 세 번 되풀이하고 끝나요.

for로 주문 튜플을 꺼내 담으면서 되풀이해요

[16]
```
for menu, price, qty in orders:
    print(menu, price, qty)
```

```
coffee 4500 2
latte 5000 1
cake 6500 3
```

orders[0]	coffee	4500	2
orders[1]	latte	5000	1
orders[2]	cake	6500	3

menu	price	qty
cake	6500	3

한 번 돌 때마다 **`menu, price, qty = orders[0]`** 같은 꺼내 담기가 저절로 일어나요.

for 앞에서 0으로 시작하고, 안에서 합계를 쌓아요

[17]
```
total = 0
for menu, price, qty in orders:
    total += price * qty
    print(menu, total)
print("합계:", total)
```

- for 앞: 한 번만 실행
- for 안: 주문마다 실행
- for 밖: 다 돈 뒤 한 번

```
coffee 9000
latte 14000
cake 33500
합계: 33500
```

시작

coffee + 4500 × 2 = 9000

latte + 5000 × 1 = 5000

cake + 6500 × 3 = 19500

total

33500

없는 번호와 없는 값은 오류가 나요

```
[18] menus[3]
```

```
IndexError: list index out of range
```

menus는 0, 1, 2번까지예요

마지막 번호는 `len(menus) - 1`

```
[19] orders[0][3]
```

```
IndexError: tuple index out of range
```

튜플 안은 0, 1, 2번까지예요

메시지에 list인지 tuple인지 나와요

```
[20] cart.remove("juice")
```

```
ValueError: list.remove(x): x not in list
```

cart에 juice가 없어요

`"juice" in cart` 로 먼저 확인

for는 콜론과 들여쓰기를 꼭 지켜요

[21]
```
for menu in menus
    print(menu)
```

```
SyntaxError: expected ':'
```

줄 끝 콜론을 빠뜨렸어요. `for menu in menus:` 처럼 끝에 콜론

[22]
```
for menu in menus:
print(menu)
```

```
IndentationError: expected an indented block after 'for' statement on line 1
```

되풀이할 줄을 들여 쓰지 않았어요. `Tab` 한 번이나 빈칸 네 개

VS Code는 콜론 뒤에 Enter를 치면 네 칸을 자동으로 들여 줘요. `Enter`

3. 영수증 만들기 · 예제 1

주문 한 줄을 칸에 맞춰 찍어요

[1]
```
orders = [("coffee", 4500, 2), ("latte", 5000, 1), ("cake", 6500, 3)]
menu, price, qty = orders[0]
print(f"{'menu':<10}{'price':>8}{'qty':>5}{'amount':>10}")
print(f"{'메뉴명':<10}{'단가':>8}{'수량':>5}{'금액':>10}")
print(f"{'메뉴명':<7}{'단가':>6}{'수량':>3}{'금액':>8}")
print(f"{menu:<10}{price:>8,}{qty:>5}{price * qty:>10,}")
```

```
menu           price  qty     amount
메뉴명               단가    수량        금액
메뉴명         단가  수량      금액
coffee     4,500     2     9,000
```

<10 >8 >5 >10

꺼내 담은 **menu**, **price**, **qty**를 칸 수에 맞춰 찍어요. 1파트 칸 맞추기와 2파트 꺼내 담기를 합쳤어요.

한글 머리글만 밀려요. 파이썬은 한글을 한 칸으로 세지만 화면에서는 두 칸이에요.

한글 글자 수만큼 칸 수를 빼요. **메뉴명**은 세 글자라 **<10** 대신 **<7**

3. 영수증 만들기 · 예제 2

for 두 줄로 영수증 한 장을 찍어요

[2]

```
print("=" * 33)
print(f"{'KANT 카페':^31}")
print("=" * 33)
print(f"{'메뉴명':<7}{'단가':>6}{'수량':>3}{'금액':>8}")
print("-" * 33)
total = 0
for menu, price, qty in orders:
    print(f"{menu:<10}{price:>8,}{qty:>5}{price * qty:>10,}")
    total += price * qty
print("-" * 33)
print(f"{'합계':<6}{total:>25,}")
print("=" * 33)
```

```
=================================
            KANT 카페
=================================
메뉴명          단가  수량        금액
---------------------------------
coffee       4,500    2     9,000
latte        5,000    1     5,000
cake         6,500    3    19,500
---------------------------------
합계                       33,500
=================================
```

같은 색끼리 짝이에요. 주문이 열 건으로 늘어도 셀은 그대로예요.

3. 영수증 만들기 · 예제 3

주문 메모 한 줄을 정리해서 다시 글자로

[3]
```
memo = "latte,cake,latte,coffee,latte"
items = memo.split(",")
items.remove("coffee")
items.append("tea")
print(f"주문 {len(items)}건, latte {items.count('latte')}잔")
items.sort()
print(" / ".join(items))
```

```
주문 5건, latte 3잔
cake / latte / latte / latte / tea
```



split으로 리스트가 되면 리스트 메서드를 다 쓸 수 있어요. 다 고친 뒤 **join**으로 다시 이어요.

정리

막히면 오류의 마지막 줄부터 읽어요

`IndexError: ... index out of range`	없는 번호예요. 마지막 번호는 len() - 1
`TypeError: not enough arguments for format string`	% 자리 수와 값 수가 달라요. f-string으로
`ValueError: unsupported format character ','`	% 에는 콤마가 없어요. f"{price:,}"
`ValueError: list.remove(x): x not in list`	없는 값을 지우려 했어요. in으로 먼저 확인
`TypeError: 'tuple' object does not support item assignment`	튜플과 문자열은 못 바꿔요. 리스트에 담아요
`SyntaxError: expected ':'`	for 줄 끝에 콜론
`IndentationError: expected an indented block ...`	되풀이할 줄을 네 칸 들여 쓰기

정리

언제 무엇을 쓰는지

글자 꺼내기, 자르기	`word[0]` `word[0:2]` `word[::-1]`
값을 문장에 끼워 넣기	`f"{menu} {qty}개"`
칸, 소수점, 콤마 맞추기	`{qty:>5}` `{pi:.2f}` `{price:,}`
값 여러 개 담고 고치기	`[ ]` `append` `insert` `remove` `pop` `sort`
묶어 두고 안 바꿀 값	`( )` `menu, price, qty = order`
글자와 리스트 오가기	`split(",")` `" / ".join(words)`
하나씩 꺼내 되풀이하기	`for menu, price, qty in orders:`

다음: 14 · 파이썬 기초 3

`in`, `not in`으로 묻기와 `scores[1][2]` 같은 2차원 리스트를 정리해요.